

Introduction to NLP

Matrix Models

Compressed out of NLP courses from **Dan Jurafsky** (Stanford), & David Bamman (Berkeley), Michael Collins (MIT & Columbia), and some online (Udemy) courses

Book: **Speech and Language Processing** by Jurafsky & Martin (3rd edition)

Revisiting Probabilistic Models

- Unsupervised Topic Modelling

Topic models

- **Unsupervised** models for discovering **hidden** “topics” or “themes” in documents

- Clusters/groups of terms that tend to occur together.

- **Input:**

- set of documents
- number of “topics” to learn

- **Output:**

- extracted topics (clusters)
- topic distribution for each document
- topic distribution for each word in a document

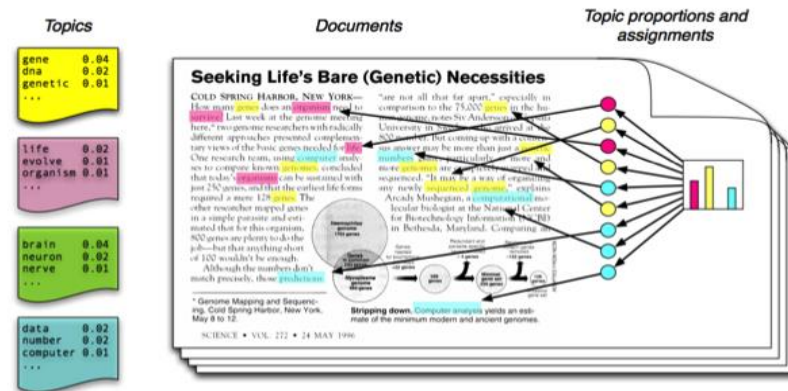
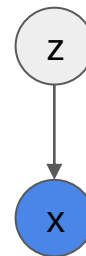


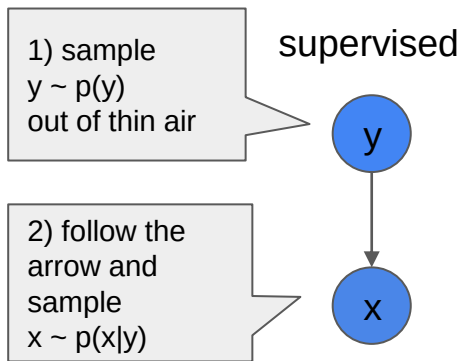
Figure source: Blei, D. M. (2012). Probabilistic topic models. *Communications of the ACM*, 55(4), 77-84.

Probabilistic topic models

- Assume a probabilistic generative process that yields the documents
 - this can be hierarchical and quite complex
- Adopts the language of probabilistic graphical models (Bayes nets)
 - Simply a visual way of writing the joint probability
 - Nodes represent variables (blue = observed, grey = latent)
 - Arrows indicate conditional relationships
 - “The probability of x is dependent on z”
- A latent variable is one that’s unobserved, either because:
 - we are predicting it (but have observed that variable for other data points)
 - it is **unobservable** (e.g., a “topic” of a document)

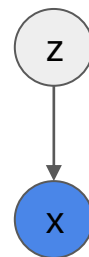


Graphical models tell a “story” of doc generation



$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

unsupervised

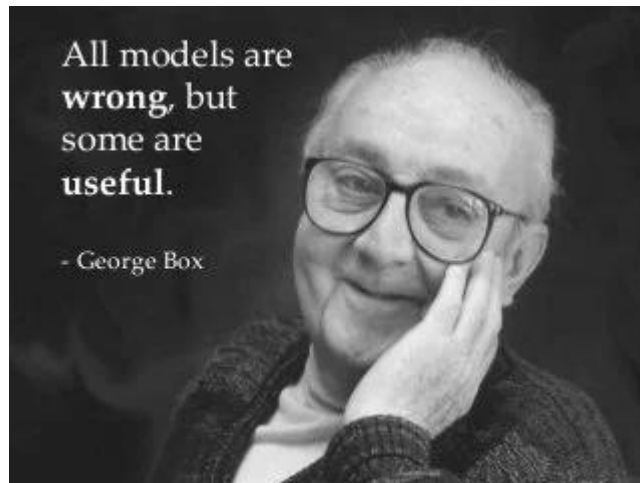
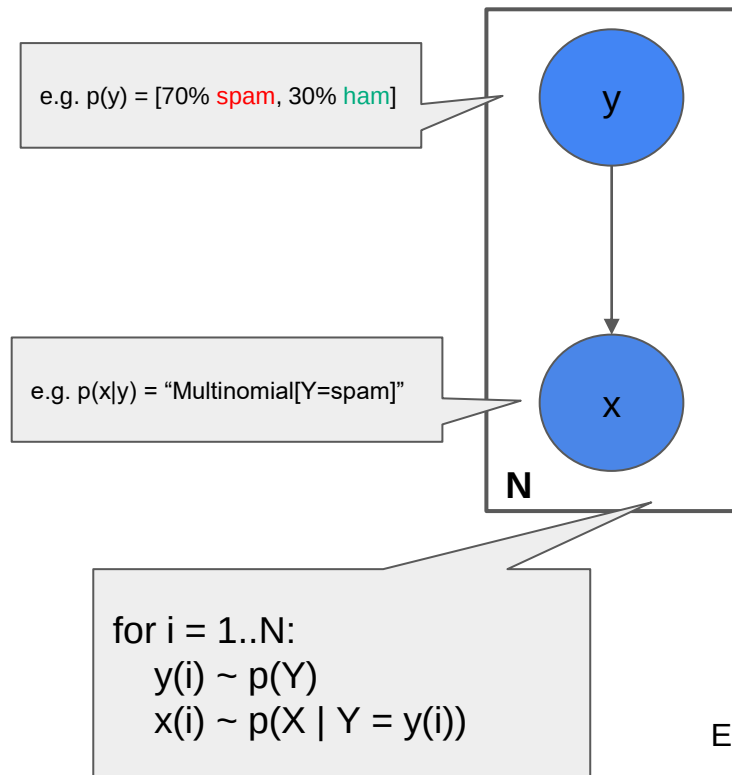


here we have to guess
(including the dimensionality)

$$P(z|x) = \frac{P(x|z)P(z)}{P(x)}$$

one needs to
resort to EM

The “story”: Plate notation



Obviously, that's not a good way to generate an email...

Email: “Hello, lottery, welcomes, deadline, 100%, viagra, inherit, ...”

Latent Dirichlet Allocation (LDA)

- The absolute classic method of choice for probabilistic topic modelling



David Blei

Professor of Statistics and Computer Science, [Columbia University](#)
E-mailová adresa ověřena na: [columbia.edu](#) - [Domovská stránka](#)

[Machine Learning](#) [Statistics](#) [Probabilistic topic models](#) [Bayesian nonparametrics](#)
[Approximate posterior infer...](#)

SLEDOVAT

Citace ZOBRAZIT VŠECHNY

	Všechny	Od 2017
Citace	106223	59185
h-index	96	80
i10-index	205	184



Andrew Ng

[Stanford University](#)
E-mailová adresa ověřena na: [cs.stanford.edu](#) - [Domovská stránka](#)

[Machine Learning](#) [Deep Learning](#) [AI](#)

SLEDOVAT

Citace ZOBRAZIT VŠECHNY

	Všechny	Od 2017
Citace	195969	114815
h-index	134	103
i10-index	295	269



Michael I. Jordan

Professor of Electrical Engineering and Computer Sciences and Professor of Statistics, [UC Berkeley](#)

E-mailová adresa ověřena na: [cs.berkeley.edu](#) - [Domovská stránka](#)

[machine learning](#) [computer science](#) [statistics](#) [artificial intelligence](#) [optimization](#)

SLEDOVAT

Citace ZOBRAZIT VŠECHNY

	Všechny	Od 2017
Citace	226651	106096
h-index	186	129
i10-index	629	480

Journal of Machine Learning Research 3 (2003) 993-1022

Submitted 2/02; Published 1/03

Latent Dirichlet Allocation

David M. Blei

*Computer Science Division
University of California
Berkeley, CA 94720, USA*

[BLEI@CS.BERKELEY.EDU](#)

Andrew Y. Ng

*Computer Science Department
Stanford University
Stanford, CA 94305, USA*

[ANG@CS.STANFORD.EDU](#)

Michael I. Jordan

*Computer Science Division and Department of Statistics
University of California
Berkeley, CA 94720, USA*

[JORDAN@CS.BERKELEY.EDU](#)

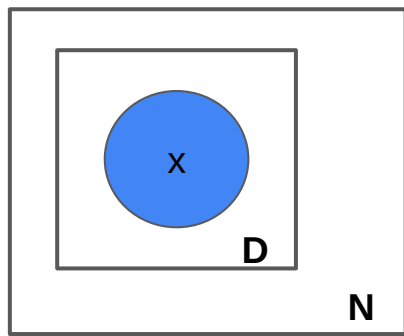
[PDF] [Latent dirichlet allocation](#)

[DM Blei](#), [AY Ng](#), [MI Jordan](#) - Journal of machine Learning research, 2003 - [jmlr.org](#)

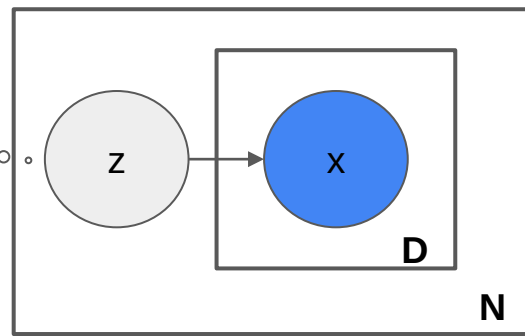
We describe **latent Dirichlet allocation** (LDA), a generative probabilistic model for collections of discrete data such as text corpora. LDA is a three level hierarchical Bayesian model, in ...

☆ Uložit Citovat Počet citací tohoto článku: 41579 Související články Všechny verze (počet: 108)

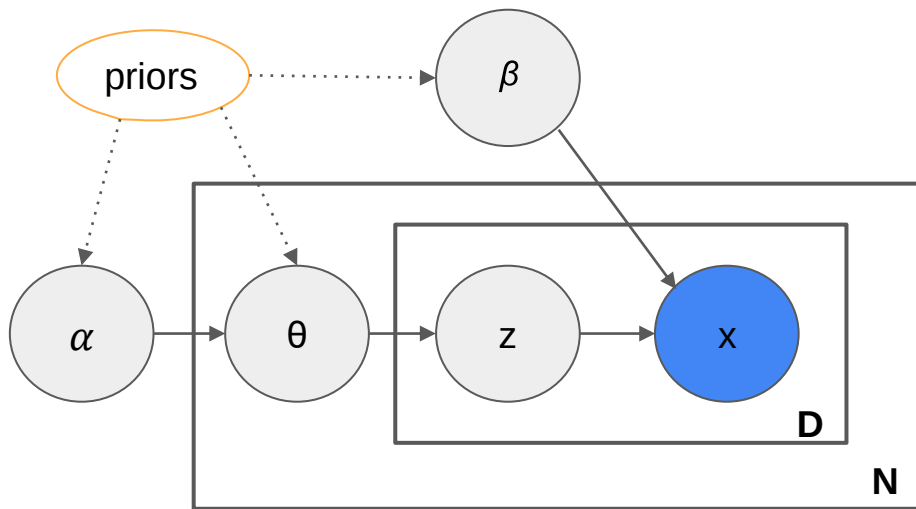
[PDF] [jmlr.org](#)



unigram



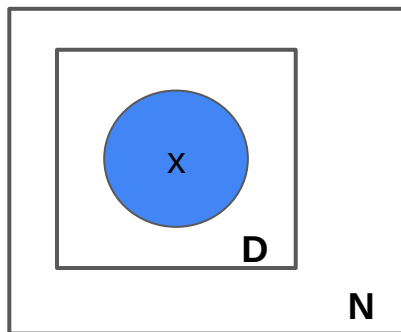
mixture of unigrams



Bayesian topic model

The “story” of corpus generation: unigrams

```
for i = 1..N:  
  for j = 1..D:  
    x(i,j) ~ p(X)
```

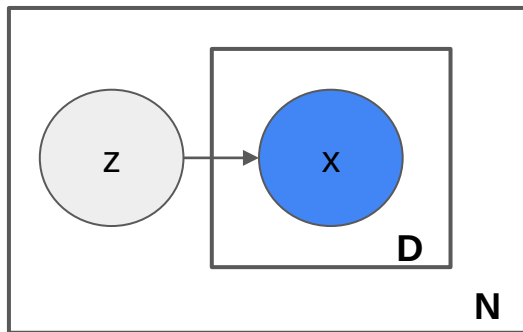


Probability of a document:

$$p(x) = \prod_{j=1}^D p(x_j)$$

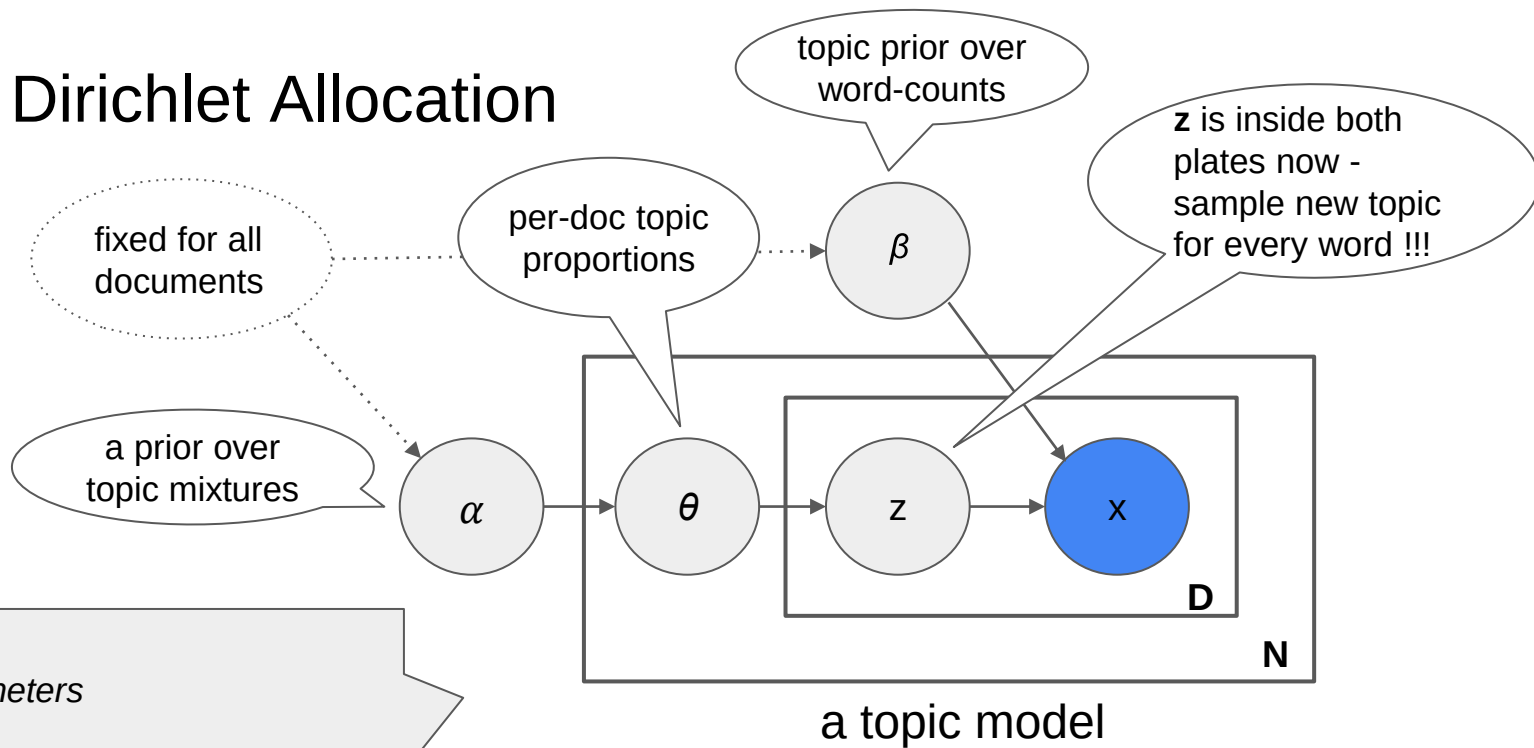
The “story” of corpus generation: mixture models

```
for i = 1..N:  
  z(i) ~ p(Z)  
  for j = 1..D:  
    x(i,j) ~ p(X | Z = z(i))
```



$$p(z, x) = p(z) \prod_{j=1}^D p(x_j | z)$$

Latent Dirichlet Allocation



$\alpha, \beta = \text{parameters}$

for $i = 1..N$:

$\theta(i) \sim \text{Dirichlet}(\alpha)$

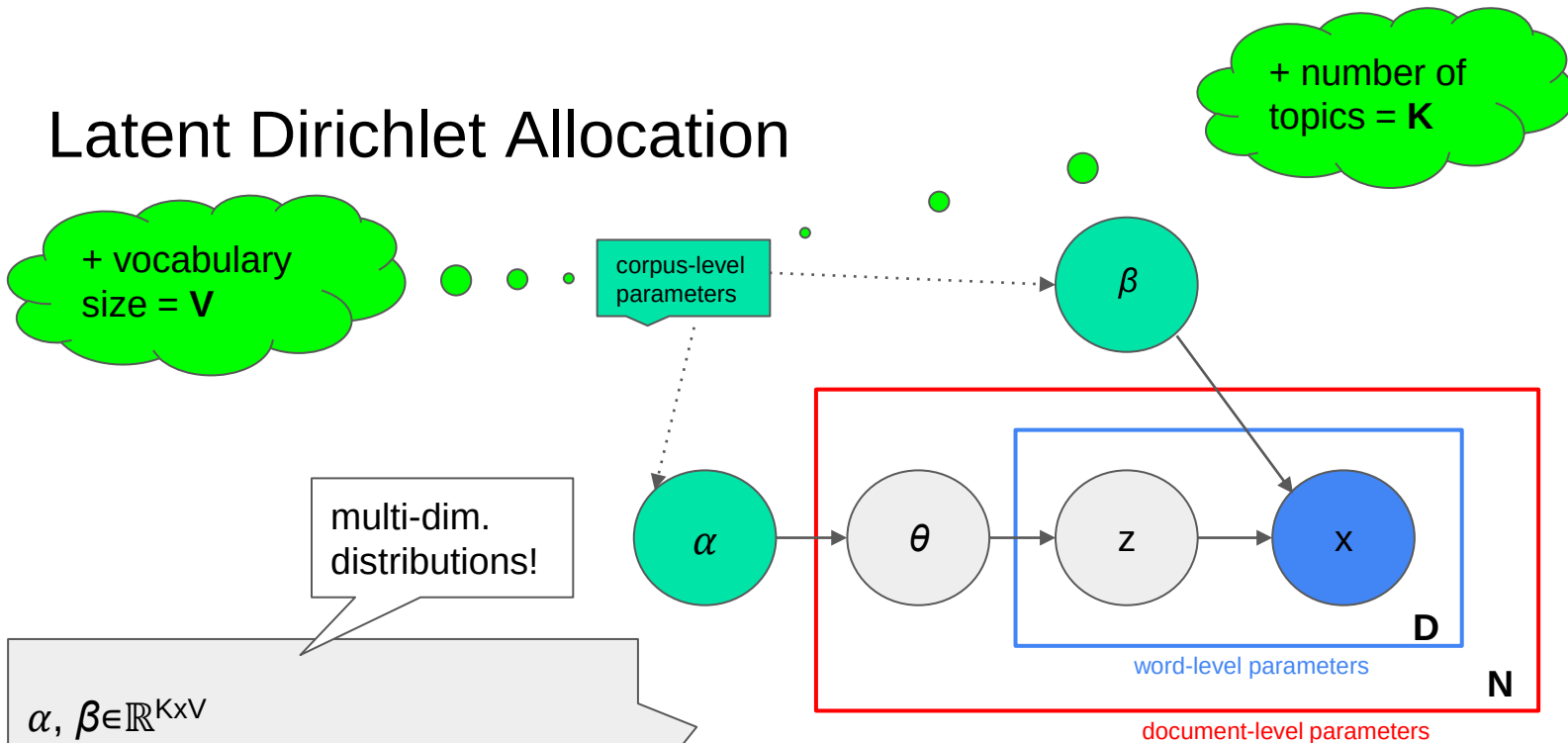
for $j = 1..D$:

$z(i,j) \sim \text{Multinom}(\theta(i))$

$x(i,j) \sim p(X | Z = z(i,j), \beta)$

$$p(\theta, z, x | \alpha, \beta) = p(\theta | \alpha) \prod_{j=1}^D p(z_j | \theta) p(x_j | z_j, \beta)$$

Latent Dirichlet Allocation



$$\alpha, \beta \in \mathbb{R}^{K \times V}$$

for $i = 1..N$:

$$\theta(i) \sim \text{Dirichlet}^K(\alpha) \quad \# \theta \in \mathbb{R}^{N \times K}$$

for $j = 1..D_i$:

$$z(i,j) \sim \text{Multinom}^K(\theta(i))$$

$$x(i,j) \sim \text{Multinom}^V(z(i,j), \beta)$$

$$p(\theta, z, x | \alpha, \beta) = p(\theta | \alpha) \prod_{j=1}^D p(z_j | \theta) p(x_j | z_j, \beta)$$

Bayesian machine learning

Under normal circumstances, θ would be a normal parameter

- e.g. a weight in a neural network

But in Bayesian ML, (almost) everything is a random variable

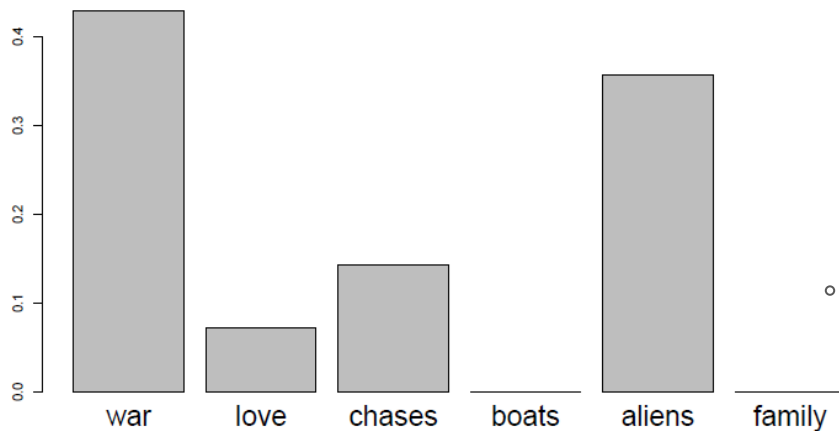
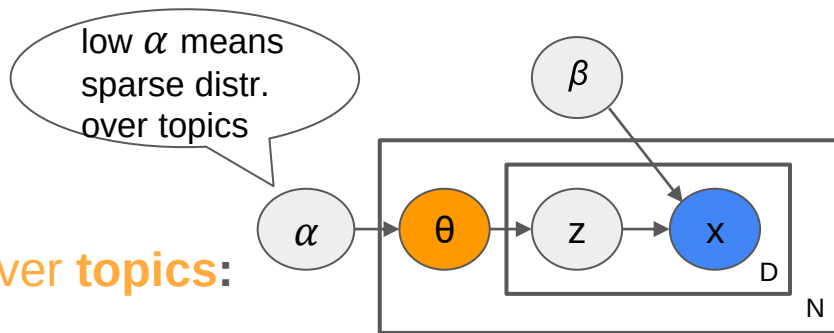
- Hence, we get a distribution over θ
- distribution with hyperparameter α
- specifically $\theta \sim \text{Dirichlet}(\alpha)$
 - typically its symmetric variant where α is a scalar (all topics a-priori equally likely)

Why Dirichlet ?

- Intuitively, Dirichlet is a distribution over positive (probability) vectors that sum up to one
 - = parameters for discrete multinomial distributions (of topics)
- Moreover, it is a **conjugate prior for the multinomial distribution** (for Bayes rule updates)

Topic model insights

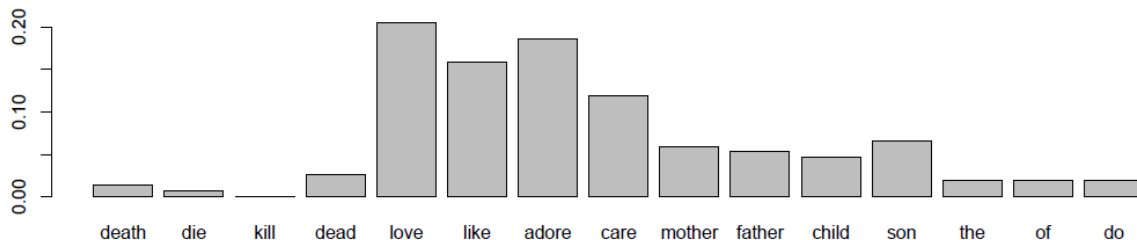
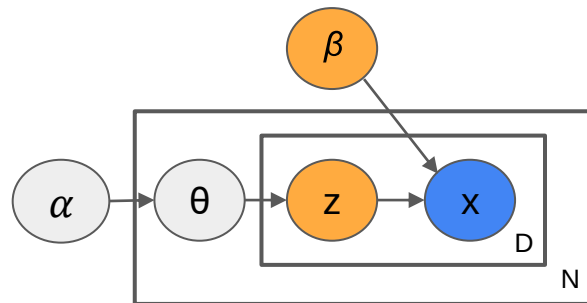
- Each document has **distribution over topics**:



topic names are "ad-hoc"

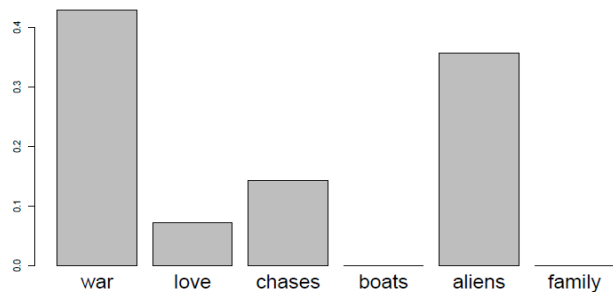
Topic model insights

- A topic is a **distribution over words**:

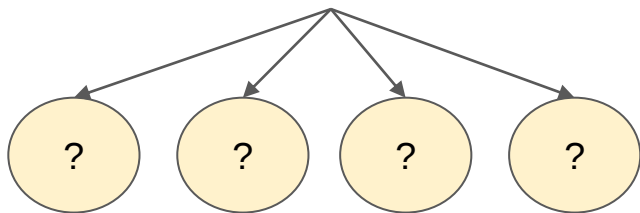


- e.g., $P(\text{"adore"} \mid \text{topic=love}) = 0.18$

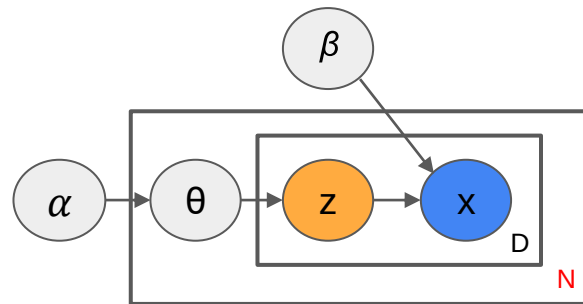
Topic model step-by-step



D=4

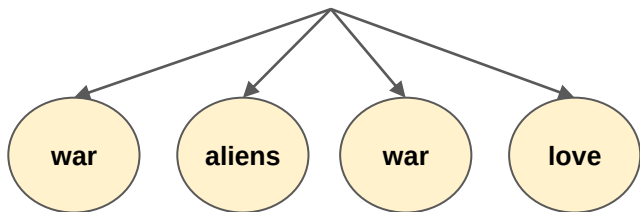
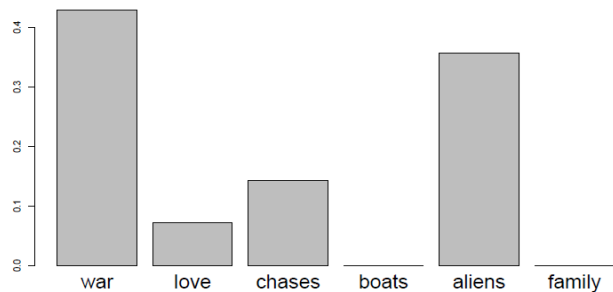


$P(\text{topic} \mid \text{topic distribution } \theta)$

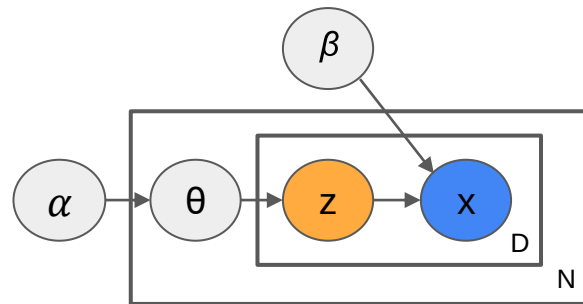


$z \sim \text{Multinomial}(\theta)$

Topic model step-by-step

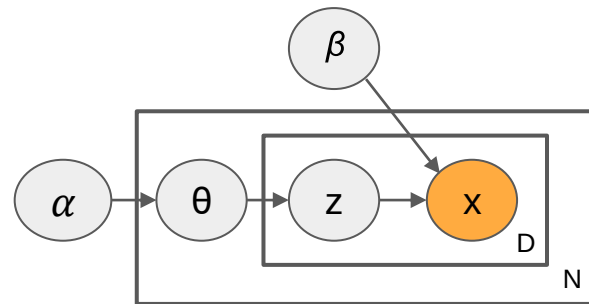
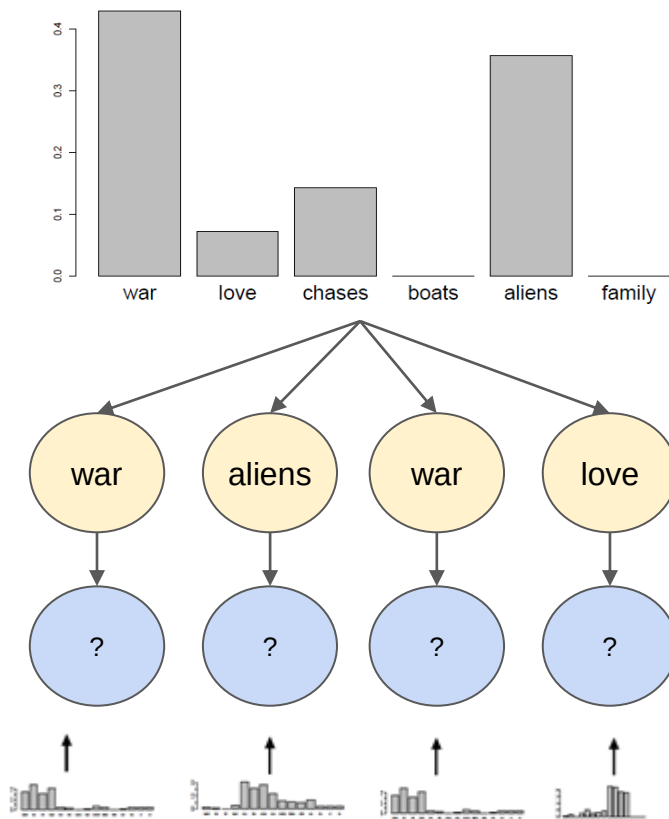


$P(\text{topic} \mid \text{topic distribution } \theta)$



$z \sim \text{Multinomial}(\theta)$

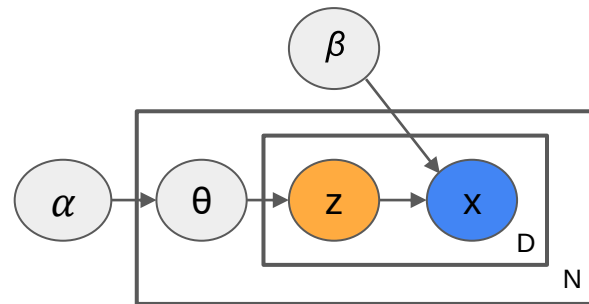
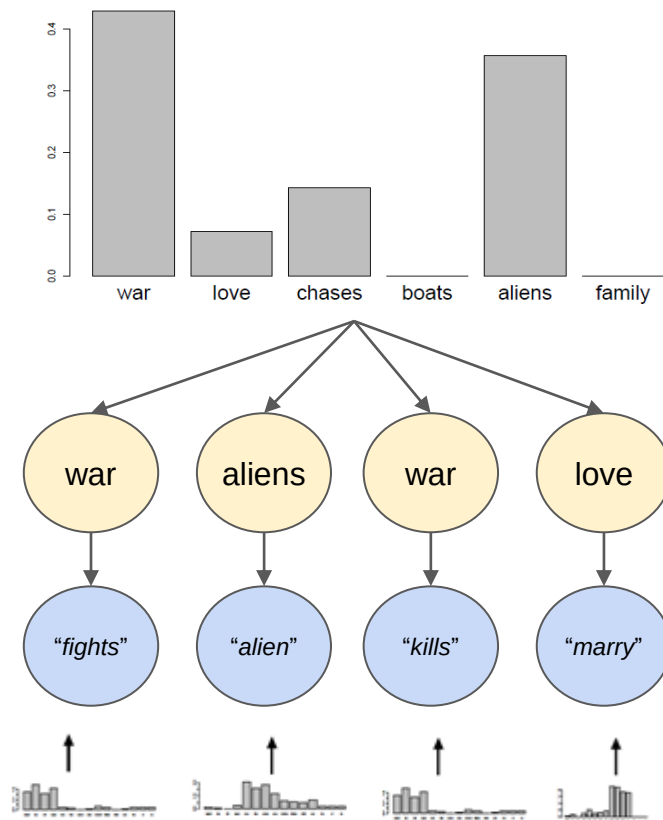
Topic model step-by-step



$$x \sim \text{Multinomial}(z, \beta)$$

$$P(\text{word} \mid \text{topic } z, \beta)$$

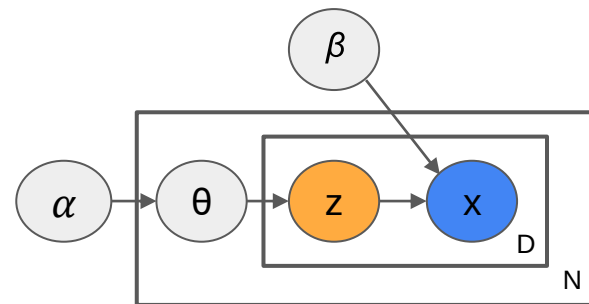
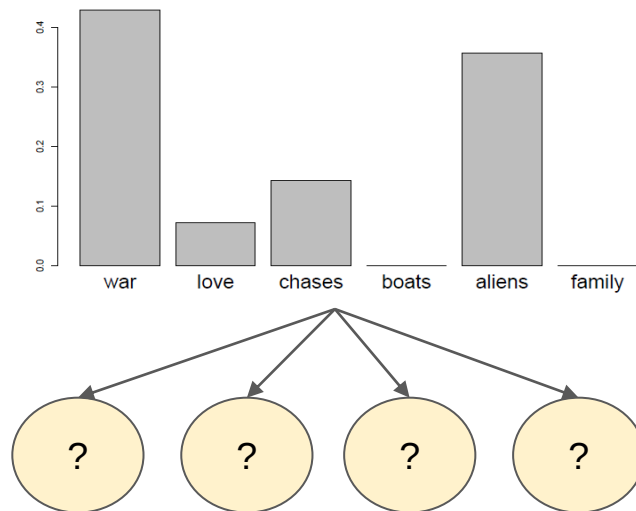
Topic model step-by-step



$$x \sim \text{Multinomial}(z, \beta)$$

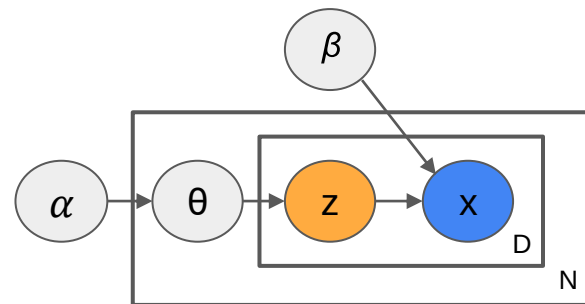
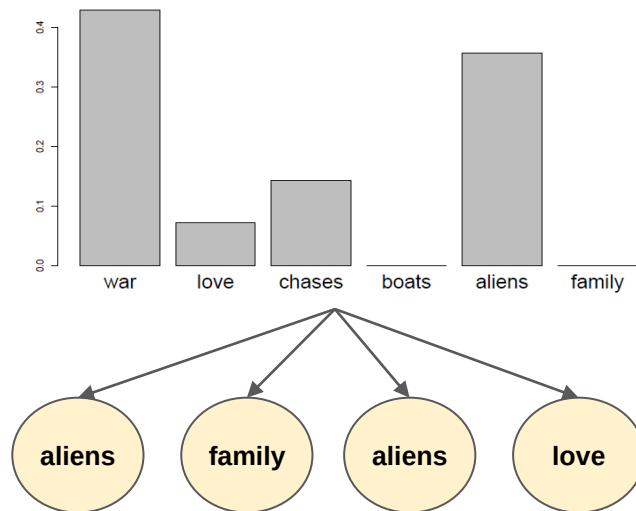
$$P(\text{word} \mid \text{topic } z, \beta)$$

Topic model step-by-step



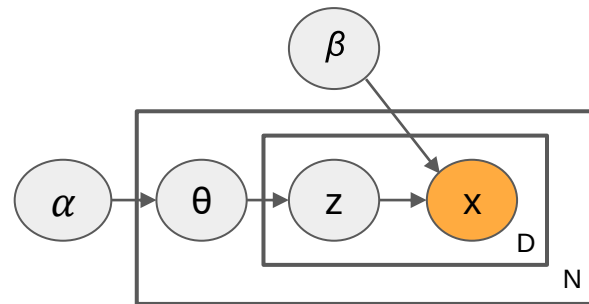
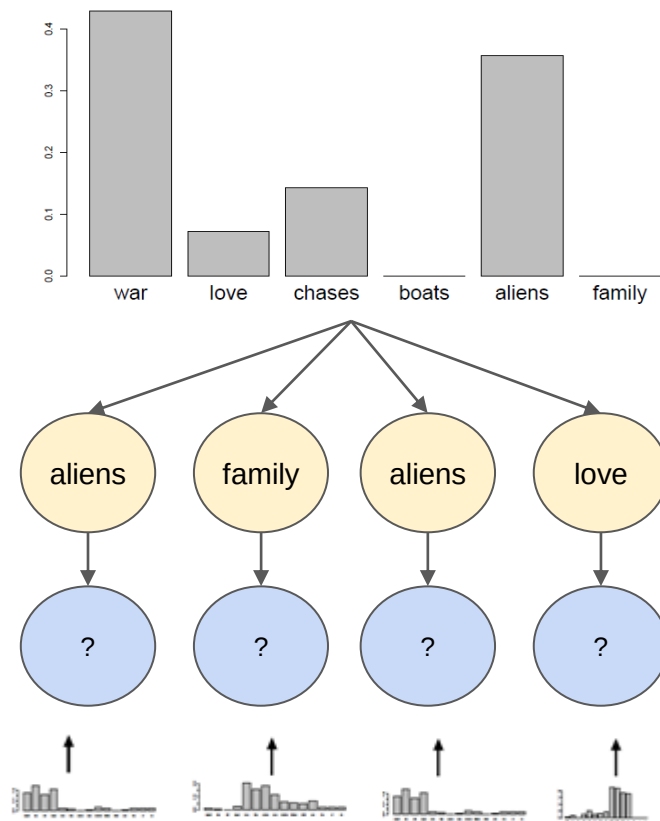
$P(\text{topic} \mid \text{topic distribution } \theta)$

Topic model step-by-step



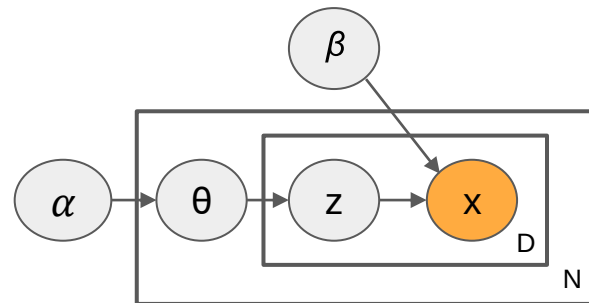
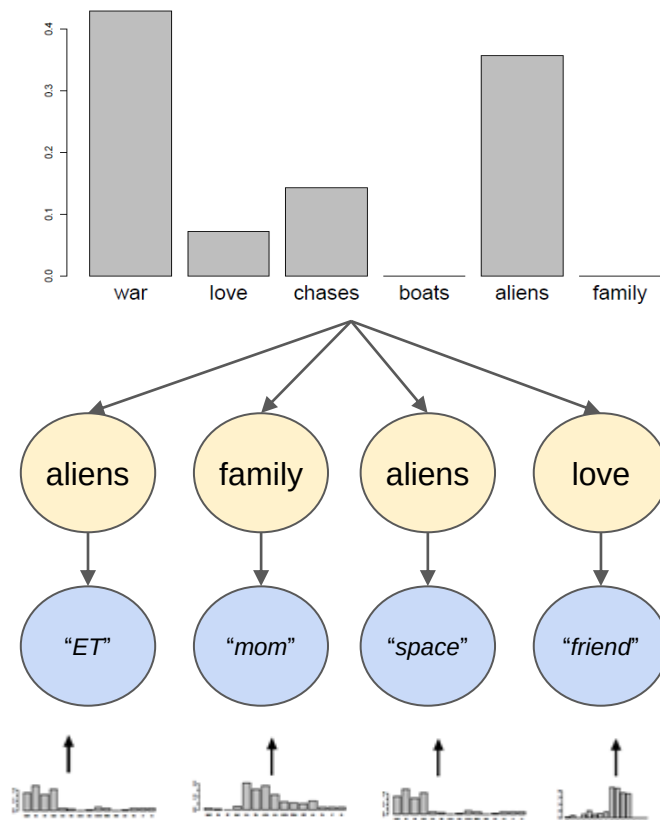
$P(\text{topic} \mid \text{topic distribution } \theta)$

Topic model step-by-step



$$P(\text{word} \mid \text{topic } z, \beta)$$

Topic model step-by-step

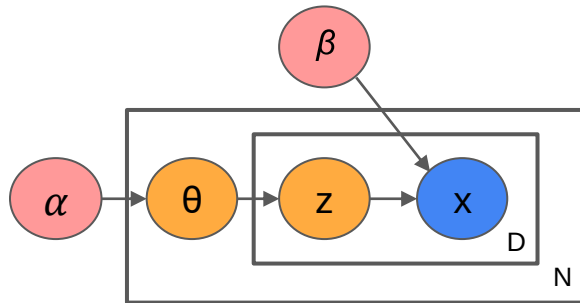


$$P(\text{word} \mid \text{topic } z, \beta)$$

Assumptions

- The only information we have are distributions of **words** across the **documents**
- No sequential information
 - topics for words are independent of each other given the set of topics for a document
- Each particular word has one topic
 - but in general we can obtain the same word from different topics!
- Every document has one topic distribution
 - Topics don't have arbitrary correlations (Dirichlet prior)

Inference = Learning



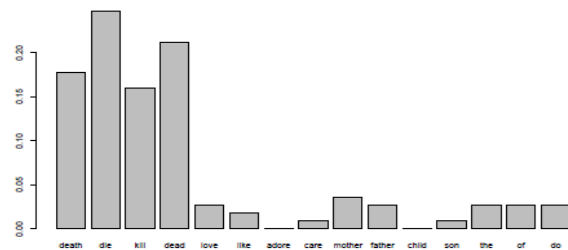
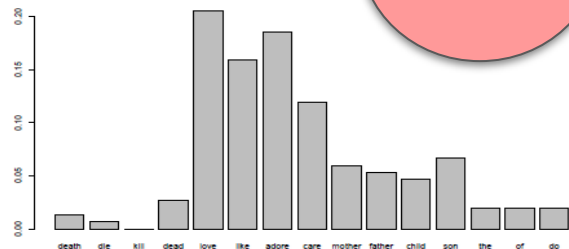
- What are the topic distributions for each document?
- What are the word distributions for each topic?
- Find the **parameters** that **maximize** the **likelihood** of the training **data**!
 - using variational EM or Gibbs sampling

Inferred Distributions: topics+words

$$\theta \in \mathbb{R}^{N \times K}$$

{album, band, music}	{government, party, election}	{game, team, player}
album	government	game
band	party	team
music	election	player
song	state	win
release	political	play
{god, call, give}	{company, market, business}	{math, number, function}
god	company	math
call	market	number
give	business	function
man	year	code
time	product	set
{city, large, area}	{math, energy, light}	{law, state, case}
city	math	law
large	energy	state
area	light	case
station	field	court
include	star	legal

$$\beta \in \mathbb{R}^{K \times V}$$



Smoothed LDA

Empirically maximizing the likelihood of training data can be problematic

Particularly in LDA, $\beta \in \mathbb{R}^{K \times V}$ is an unseen but **fixed** value

- i.e., it is a **point estimate** with no distribution
- this means zero probability for unseen words!

What can we do? You already know what..

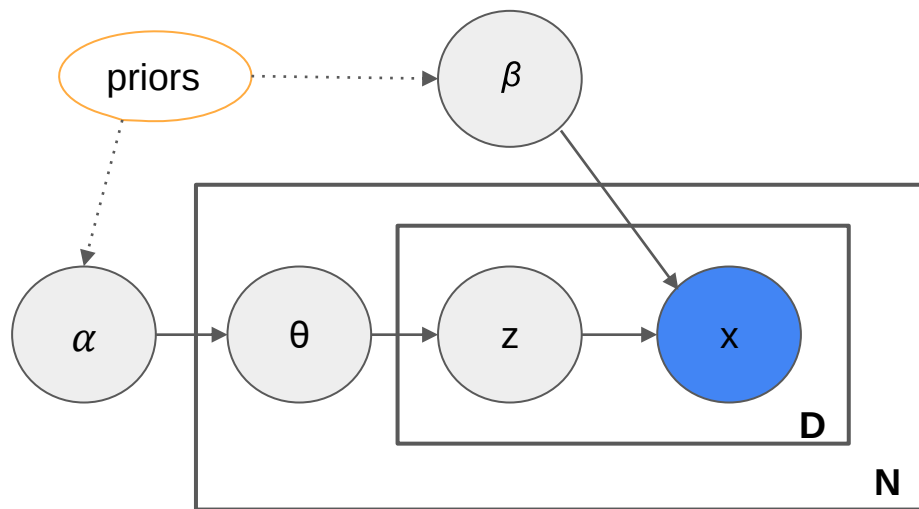
- Apply **smoothing**!
 - e.g. +1 (Laplace) that we discussed
 - actually used in practice, but rather ad-hoc

A more principled solution?

- **go more Bayesian!**

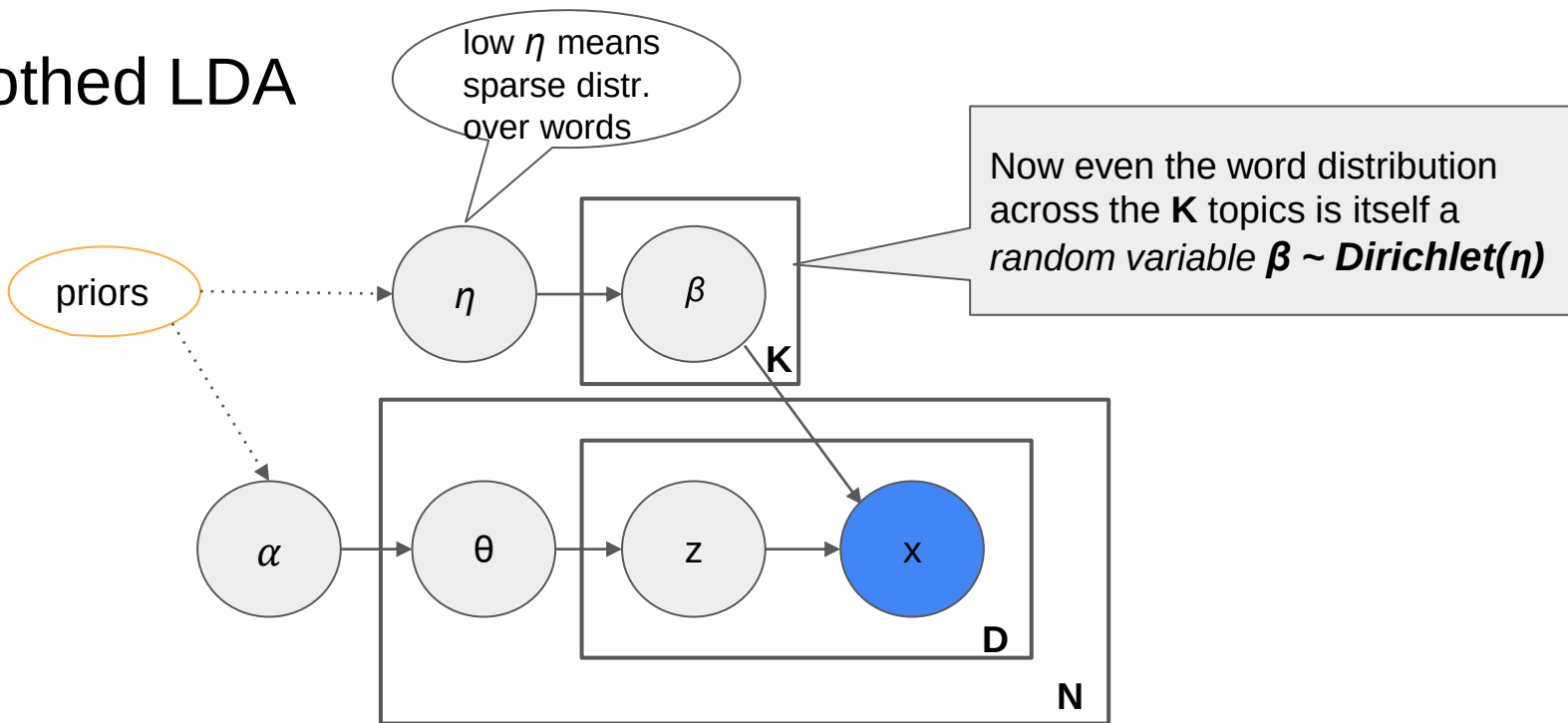


Smoothed LDA



A Bayesian topic model

Smoothed LDA



A **smoothed** Bayesian topic model

- With this η we now have a **Dirichlet** prior for all the words (even unseen in the corpus!)

How to use in practice

a) Implement your own LDA and variational EM...

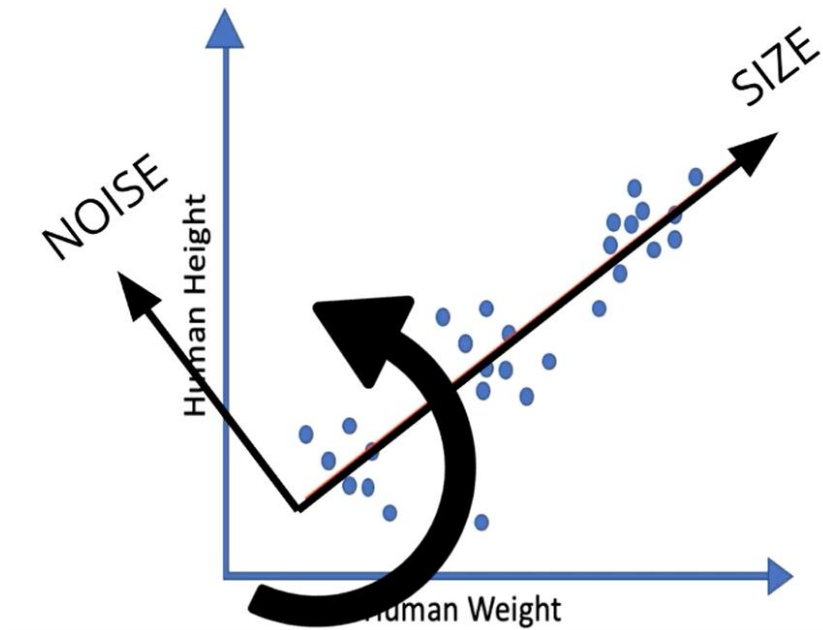
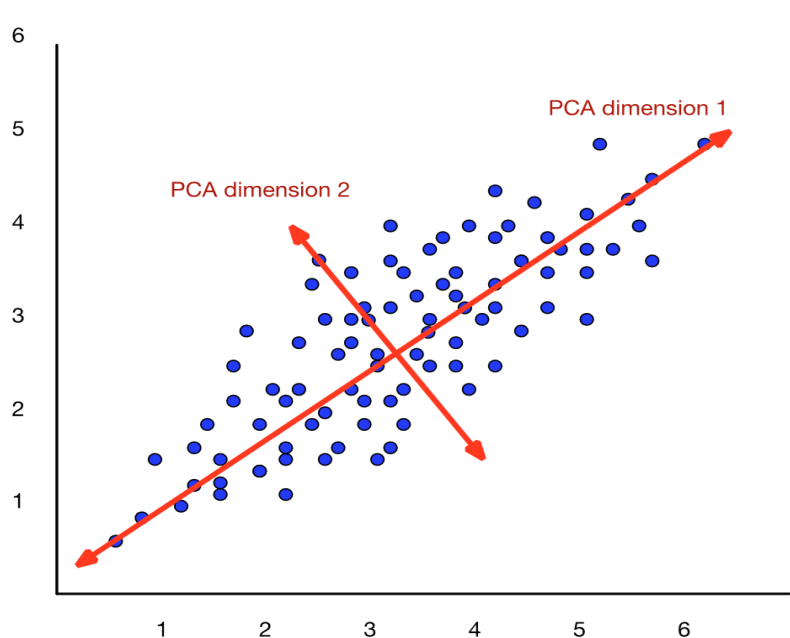
b) **from** sklearn.decomposition **import** LatentDirichletAllocation

- not to be confused with another LDA = Linear Discriminant Analysis
 - used to be in sklearn.lda.LDA
 - now: sklearn.discriminant_analysis.LinearDiscriminantAnalysis
- Other popular libraries:
 - Gensim
 - from Radim Rehurek
 - Spacy
- In the next lecture, we will see a complementary approach with matrix decompositions

Revisiting Vector models

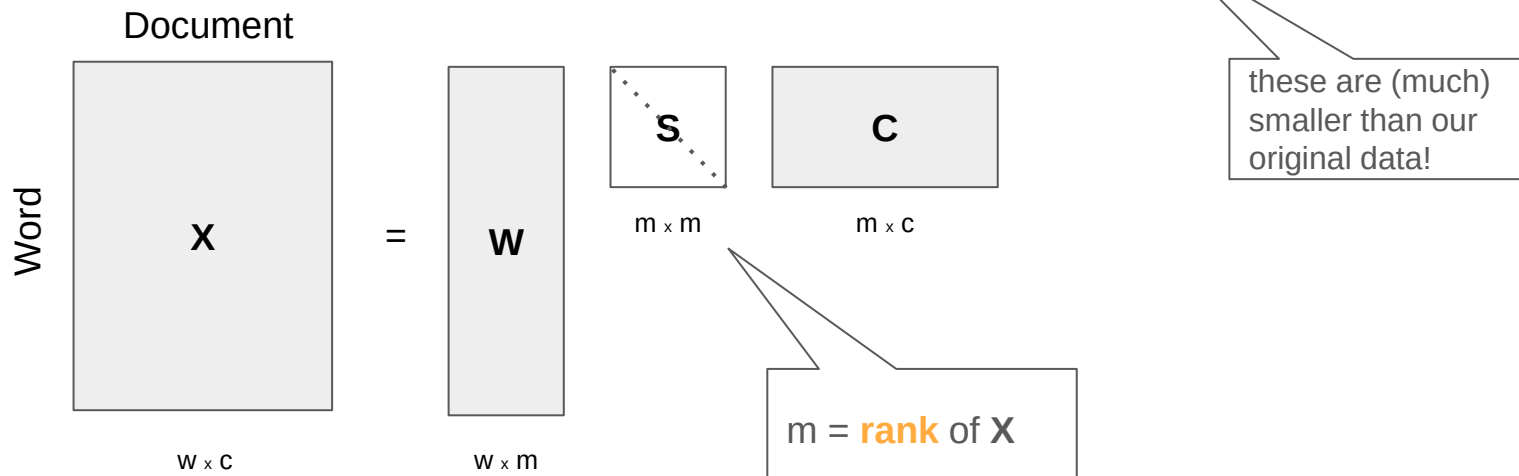
- SVD Factorization of the Term-Document Matrix

Dimensionality reduction



Singular Value Decomposition (SVD)

Any $w \times c$ matrix X equals the product of 3 matrices: $X = W S C$



- note that $\text{rank}(X) \leq \min(w, c)$
- reveals the “true” dimensionality of our data

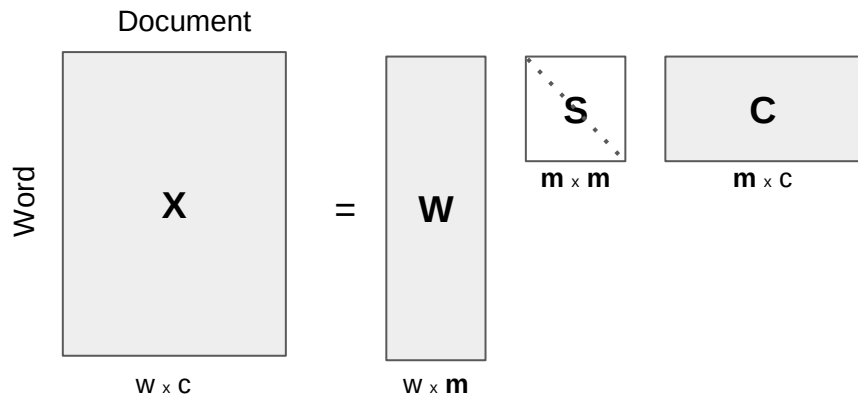
Singular Value Decomposition

Any $w \times c$ matrix X equals the product of 3 matrices: $X = W S C$

- **W** ($w \times m$): rows corresponding to original, but **m** columns represents a dimension in a new latent space, such that
 - **m** column vectors are orthogonal to each other
 - Columns are ordered by the amount of variance in the dataset each new dimension explains
- **S** ($m \times m$): **diagonal** $m \times m$ matrix of **singular values** expressing the importance of each dimension.
- **C** ($m \times c$): columns corresponding to original, but **m** rows corresponding to the singular values

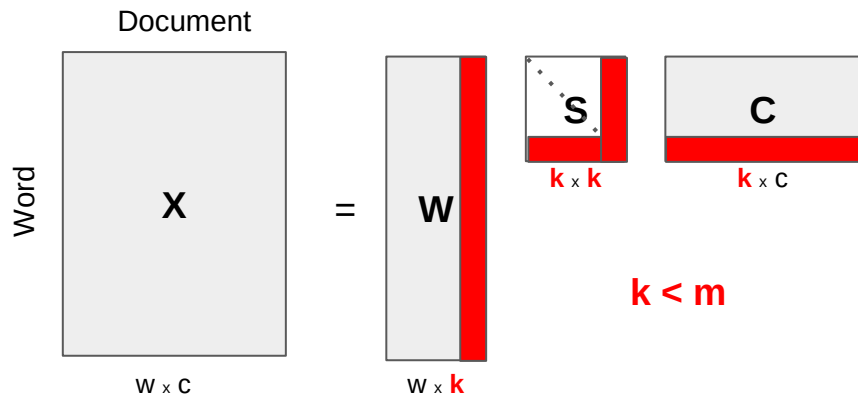
Truncated Singular Value Decomposition

- Often, m is not small enough
- Instead of keeping all m dimensions, we just keep the **top k** singular values.
 - Let's say 300.
- The result is a least-squares approximation to the original X
- Each row of W is:
 - k -dimensional vector
 - Representing word W



Truncated Singular Value Decomposition

- Often, m is not small enough
- Instead of keeping all m dimensions, we just keep the **top k** singular values.
 - Let's say 300.
- The result is a least-squares approximation to the original X
- Each row of W is:
 - k -dimensional vector
 - Representing word W

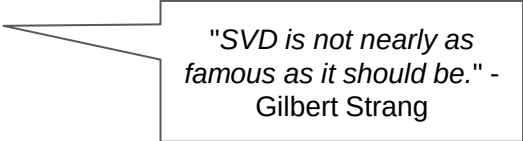


Revisiting topic modelling

- topic modelling as matrix factorization

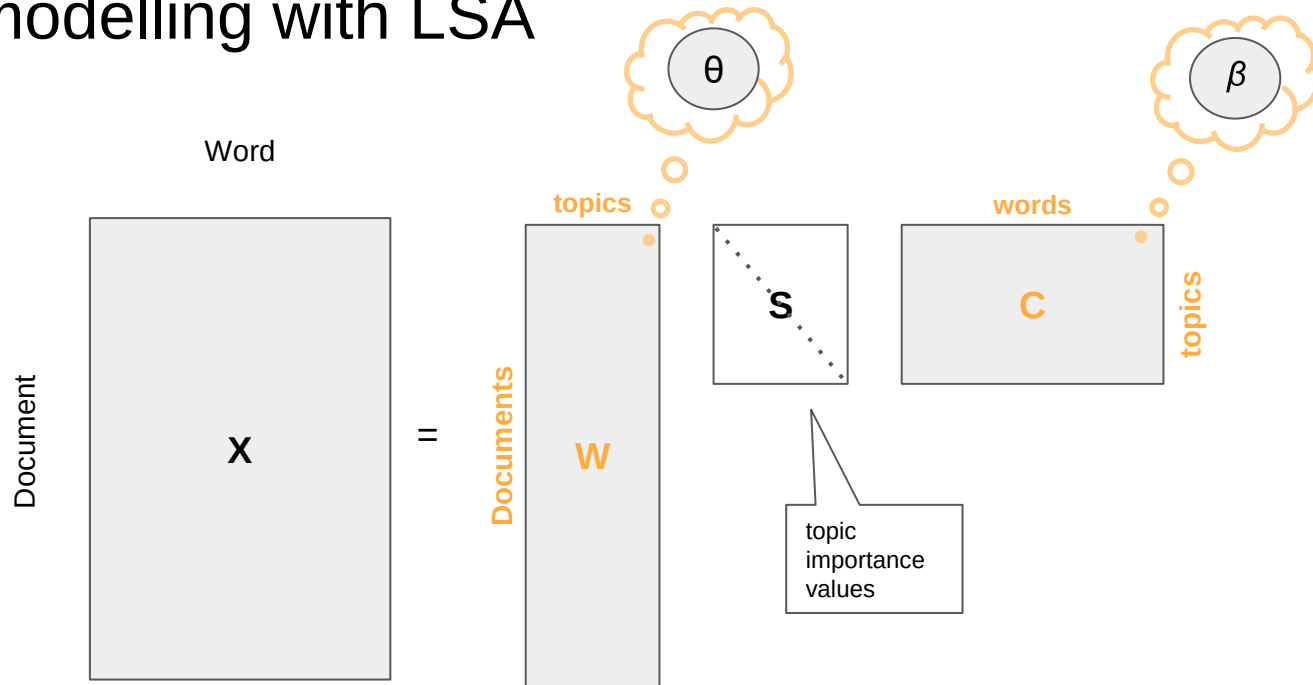
Latent Semantic Analysis

- LSA is often referred to as “topic modelling” itself
- SVD applied to the Document-Term matrix = **Latent Semantic Analysis**
 - 300 dimensions are commonly used for **k**
- The cells are commonly weighted by **TF-IDF**
- **k** topics = **k** latent dimensions
- we expect the word distr. across the topics to be distinct/orthogonal
 - this is exactly what SVD does!



"SVD is not nearly as famous as it should be." - Gilbert Strang

Topic modelling with LSA

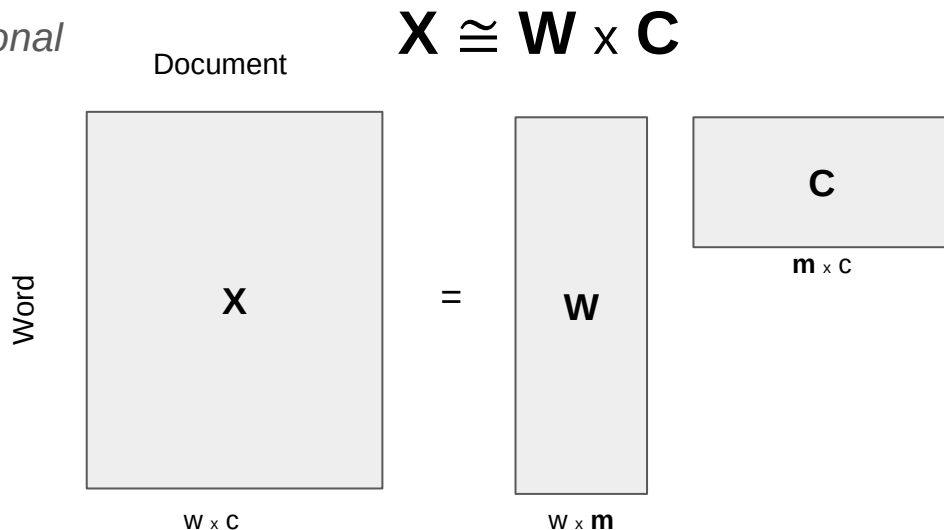


= the same output format as we have seen from LDA!

Non-negative Matrix Factorization

Alternative decomposition: **Non-negative Matrix Factorization** (NNMF)

- Idea: constrain the latent topics to be ***non-negative***
 - rather than constraining to be *orthogonal*
- This is easier to **interpret** the topics
 - W ~ amount of words in topics
 - C ~ amount of topics in documents

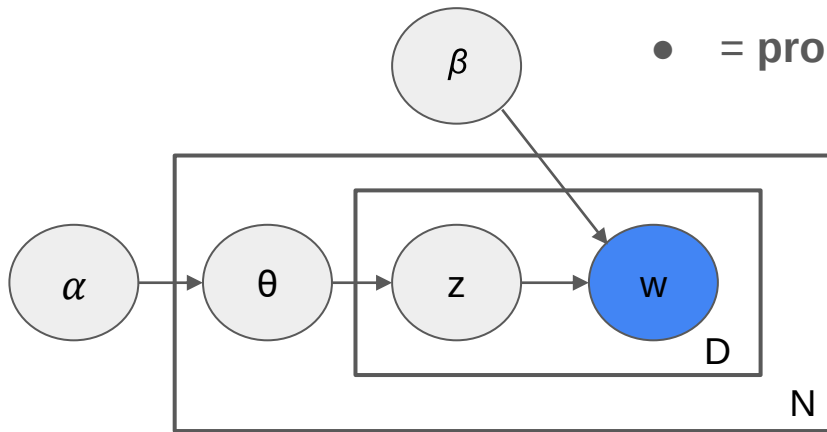


Non-negative Matrix Factorization

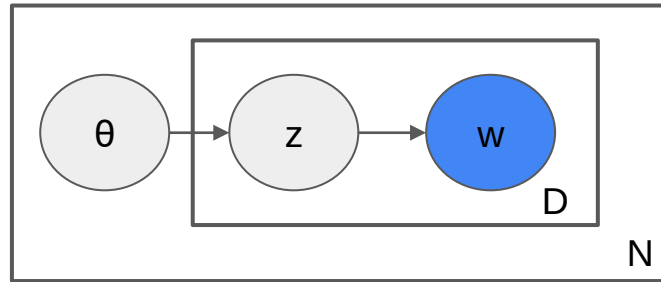
NNMF is only approximate

- different optimization criteria for the $\mathbf{X} \cong \mathbf{W} \times \mathbf{C}$ problem
- with Kullback-Leibler divergence $KL(\mathbf{X}; \mathbf{W} \times \mathbf{C})$

- = probabilistic LSA (**pLSA**)



LDA = Bayesian



pLSA = not Bayesian

Revisiting Vector Semantics

- **Word embeddings** via **SVD** of **Term-Term** matrix

SVD applied to Term-Term matrix

- ...let's return to the PPMI **Term-Term** matrices
 - can we apply SVD to them?

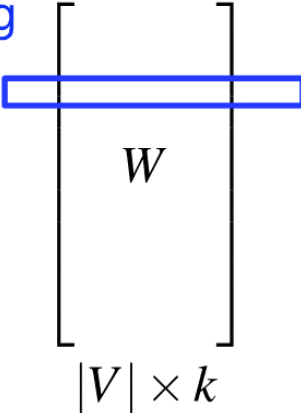
$$\begin{bmatrix} X \\ |V| \times |V| \end{bmatrix} = \begin{bmatrix} W \\ |V| \times |V| \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_V \\ |V| \times |V| \end{bmatrix} \begin{bmatrix} C \\ |V| \times |V| \end{bmatrix}$$

- simplifying assumption: the matrix has rank $|V|$

Truncated SVD produces **embeddings**

$$\begin{bmatrix} X \\ |V| \times |V| \end{bmatrix} = \begin{bmatrix} W \\ |V| \times k \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_k \end{bmatrix} \begin{bmatrix} C \\ k \times |V| \end{bmatrix}$$

embedding
for
word i



W

$|V| \times k$

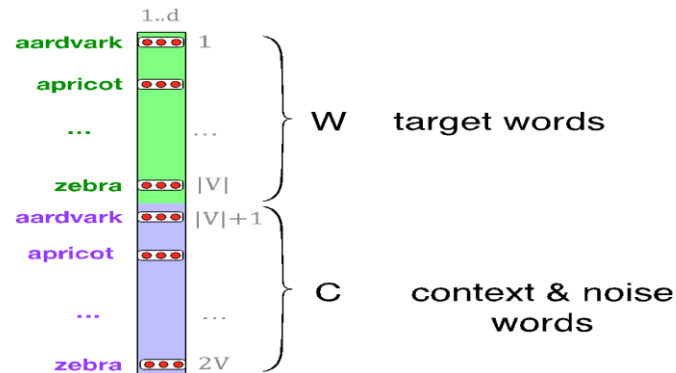
- Dense SVD embeddings vs. sparse PPMI matrices
- generally better at tasks requiring **word similarity**
 - Denoising: low-order dimensions represent noise
 - Truncation may help the models generalize better to unseen data.

Revisiting word2vec (SGNS)

- We train a “probabilistic” classifier, given:
 1. a test target word $\mathbf{w}$
 2. its context window of L words $\mathbf{c}_{1:L}$
- Estimate probability that $\mathbf{w}$ occurs in this window based on similarity of $\mathbf{w}$ (embeddings) to the $\mathbf{c}_{1:L}$ (embeddings).

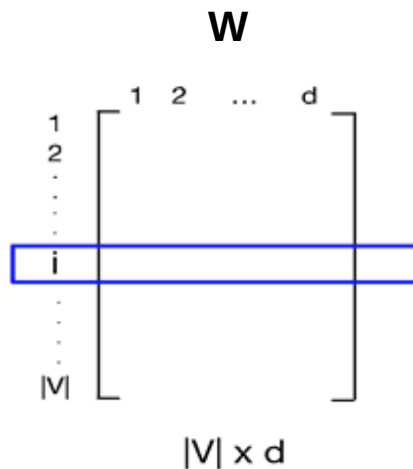
We need to learn the embeddings:

$\theta =$



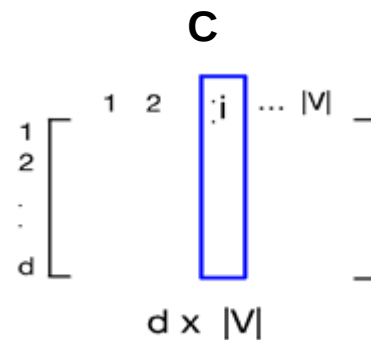
2 types of word embeddings

$$\begin{bmatrix} X \\ |V| \times |V| \end{bmatrix} = \begin{bmatrix} W \\ |V| \times k \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_k \end{bmatrix} \begin{bmatrix} C \\ k \times |V| \end{bmatrix}$$



Word embedding w_i , in the input matrix **W**

Row i of the input matrix W is the $1 \times d$ vector embedding w_i for word i in the vocabulary.



Context embedding c_i , in output matrix **C**

Column i of the output matrix C' is a $d \times 1$ vector embedding c_i for word i in the vocabulary.

Relation between skip-grams and PMI!

- Note that if we multiply WC^T
 - We get a $|V| \times |V|$ matrix M , where each entry m_{ij} corresponds to some association between input word i and output word j
 - It can be shown that skip-gram reaches its optimum just when this matrix M is a shifted version of the PMI matrix:

$$WC^T = M^{\text{PMI}} - \log k$$

instead of truncating at 0 (PPMI), we shift by k (negative sampling)

- So, skip-gram word2vec is implicitly factoring a shifted version of the PMI matrix into the two embedding matrices!

Problems with SVD

Problems with SVD

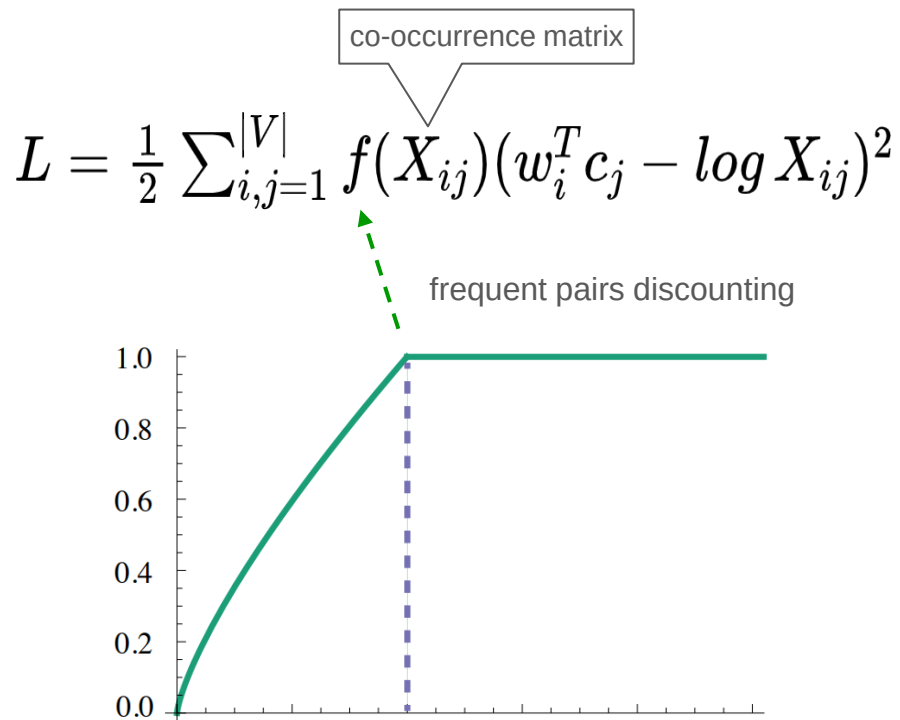
- Computational cost scales quadratically, for $w \times c$ matrix:

$O(wc^2)$ operations (when $c < w$)

- → Bad for millions of words or documents!
- Hard to incorporate new words or documents
- Different learning regime than common ML models

GloVe

- Can we combine these 2 approaches?
 - To make use of the co-occurrence counts
 - while avoiding the full matrix decomposition
- **GloVe** = Global Vectors
 - introduces a custom loss fcn **L**
- We iterate through all pairs of words in **X**
 - optimizing one co-occurrence **count** at a time
- No need to iterate the large text corpus
 - just through the aggregated counts
- Fast training, good even with small data



Embedding space geometry: GloVe

Nearest words to
frog:

1. frogs
2. toad
3. litoria
4. leptodactylidae
5. rana
6. lizard
7. eleutherodactylus



litoria



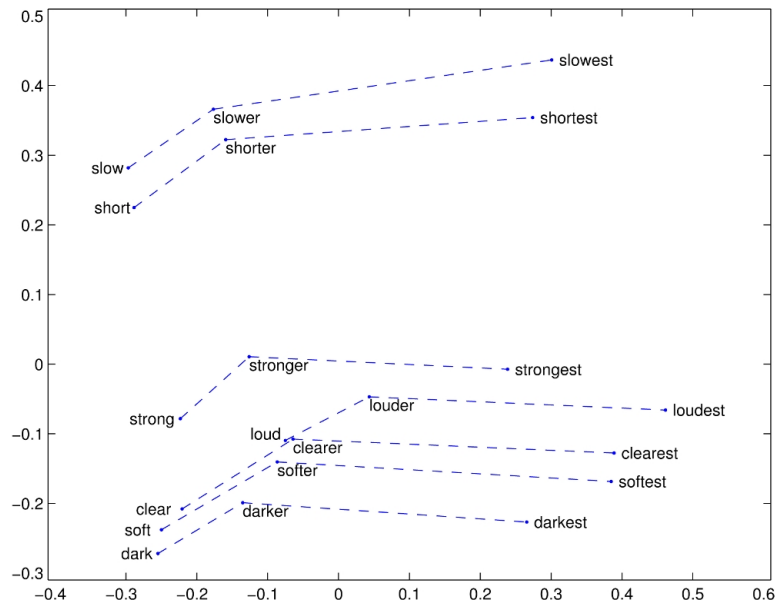
leptodactylidae



rana



eleutherodactylus

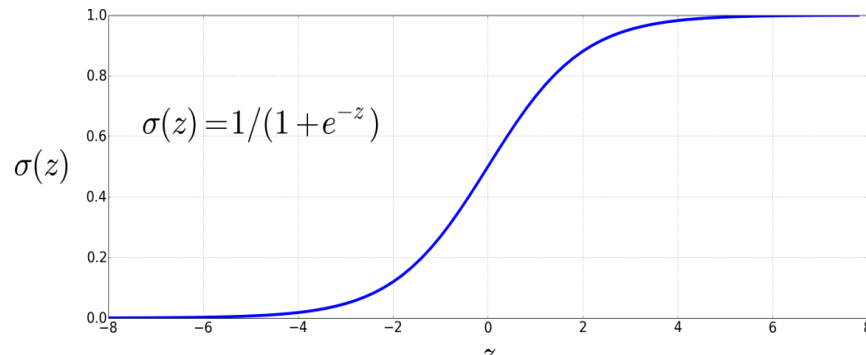
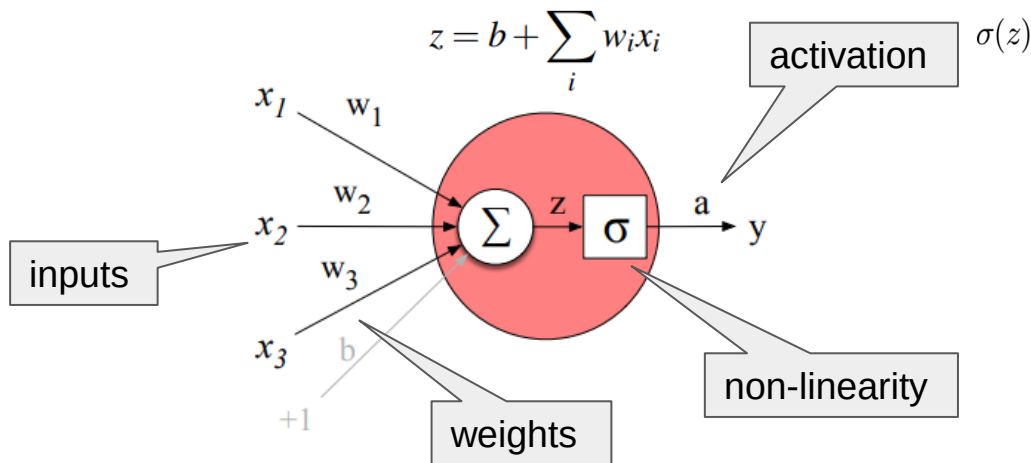


Matrix models

- Introduction to **Neural Models**

A Neuron

- Basically a **Logistic Regression**
 - also used in NLP a lot

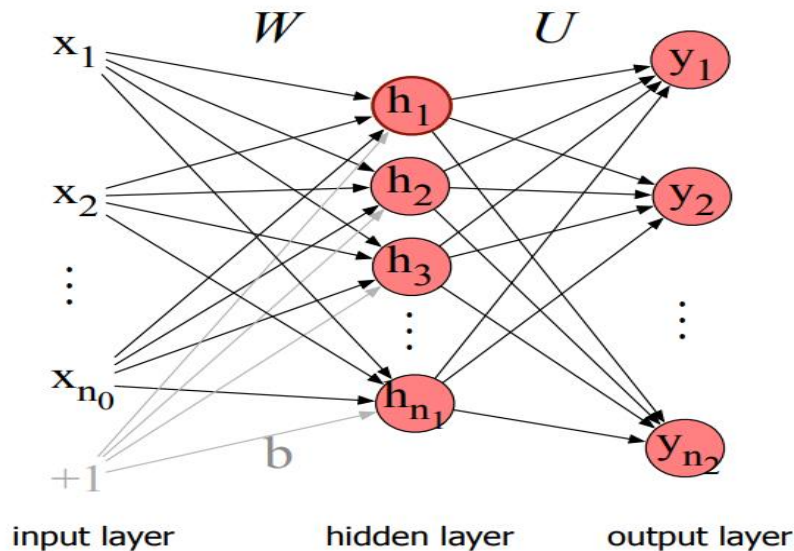
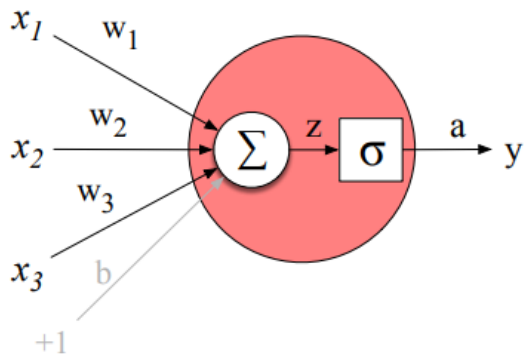


$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))}$$

Feed-Forward Neural Networks

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- Stacking the neurons into “hidden” layers
 - with which we move to **matrix notation**
- You are already familiar with these models from RPZ, SSU, UI, ...



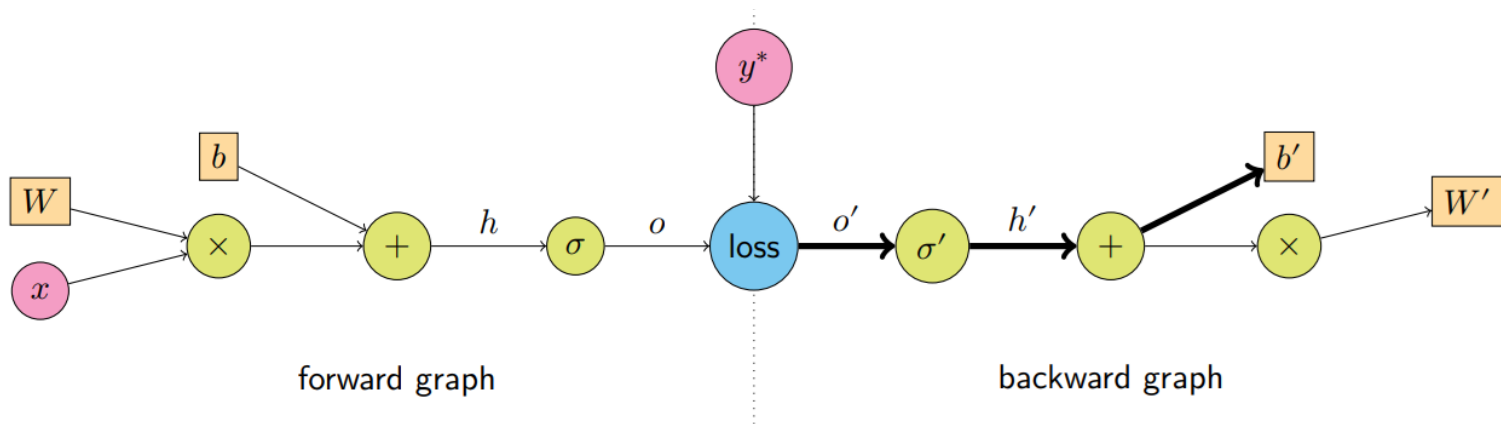
Computational graph: logistic regression

source: <https://ufal.mff.cuni.cz/jindrich-libovicky>

Logistic regression:

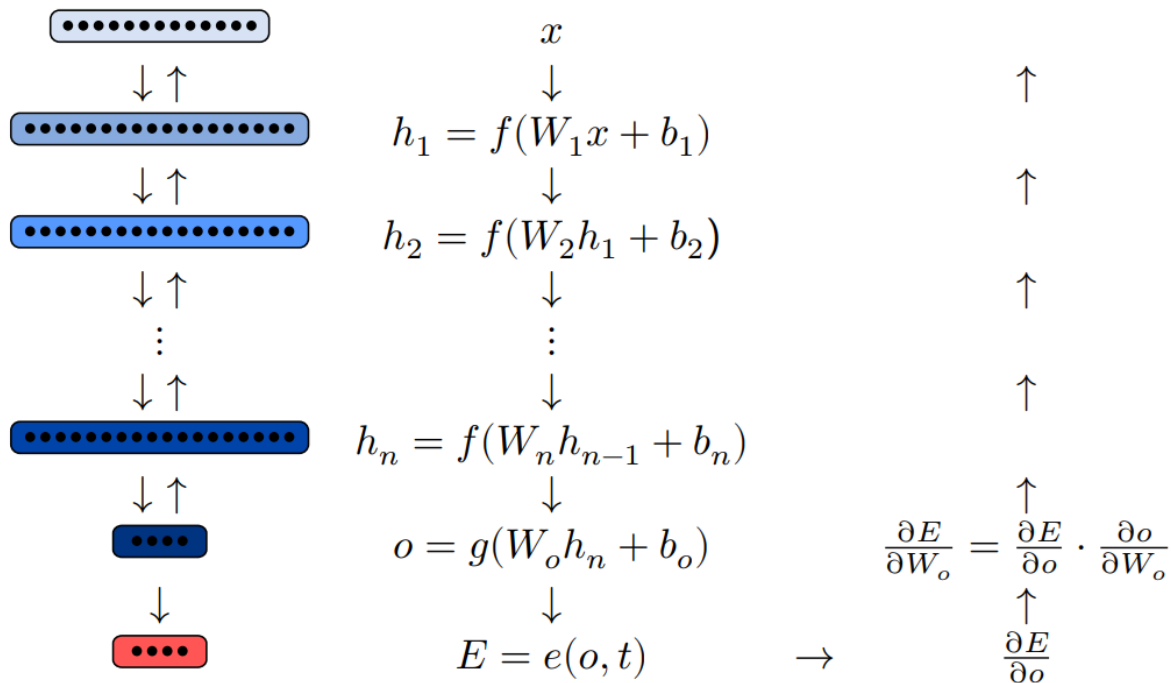
$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

Computation graph:



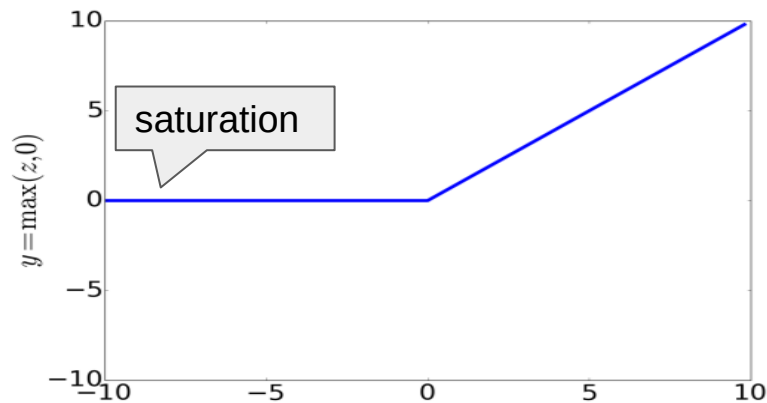
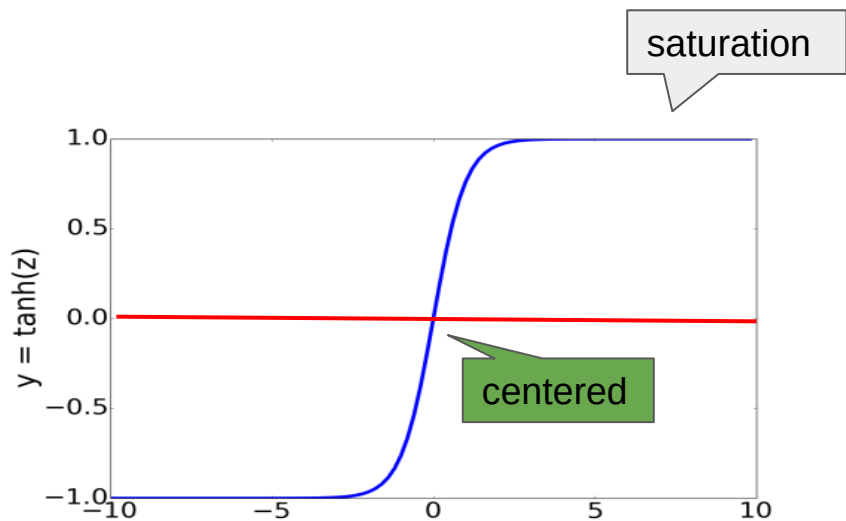
Feed-Forward Neural Networks

source: <https://ufal.mff.cuni.cz/jindrich-libovicky>



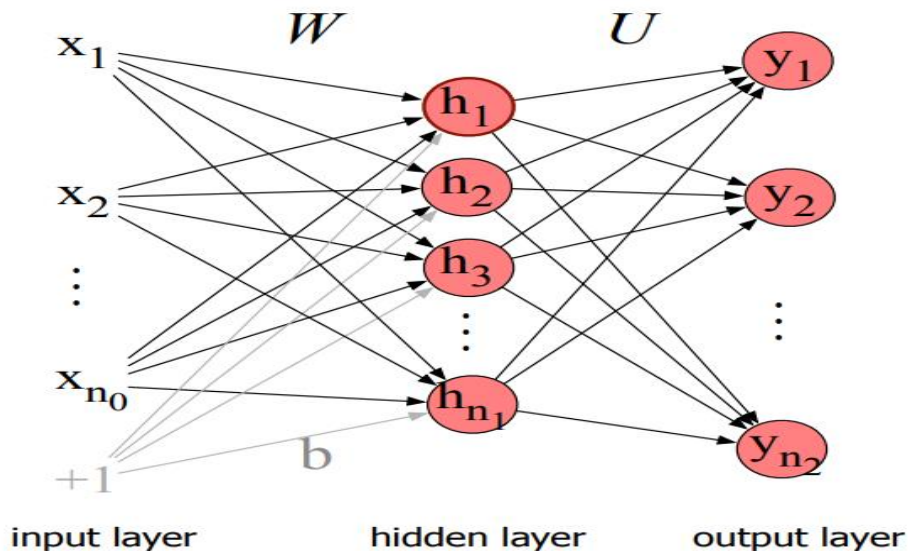
Activations

- Sigmoid non-linearity has some problems in practice
 - deviation + saturation
- Some better choices:



Feed-Forward Neural Networks

- To obtain multi-class probabilities, we should **normalize** the values



$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$$

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

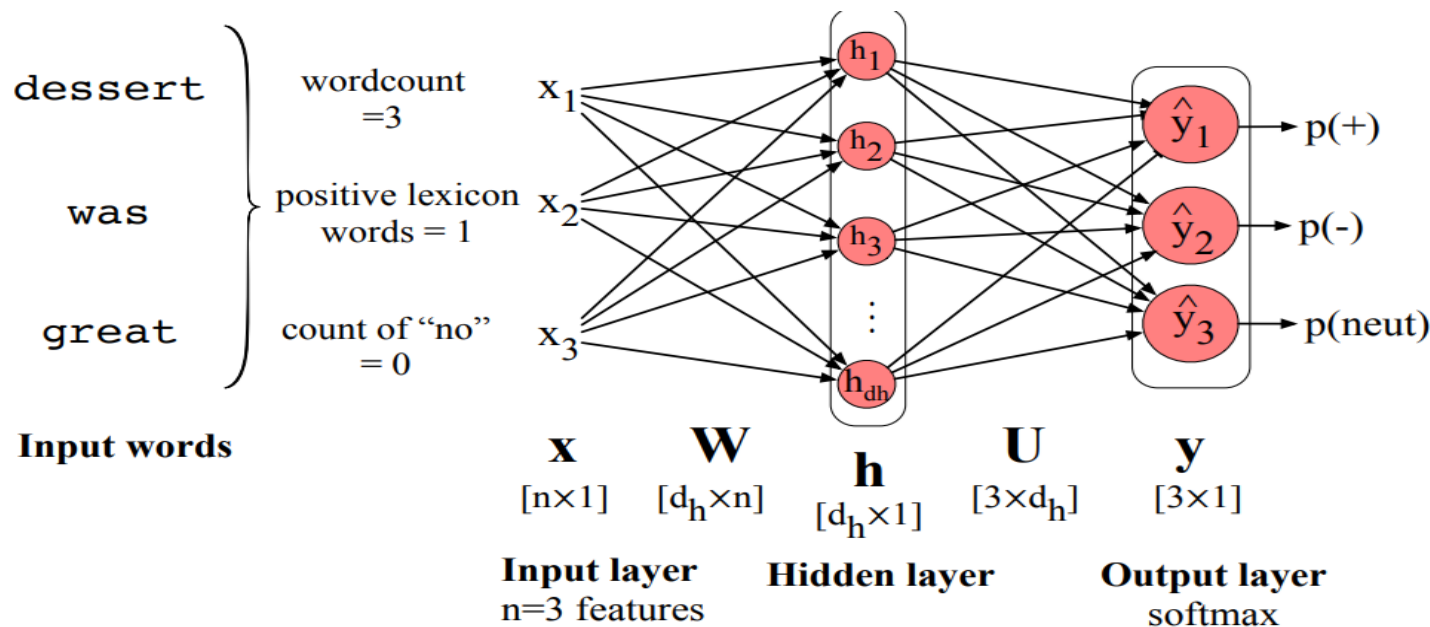
$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^d \exp(\mathbf{z}_j)} \quad 1 \leq i \leq d$$

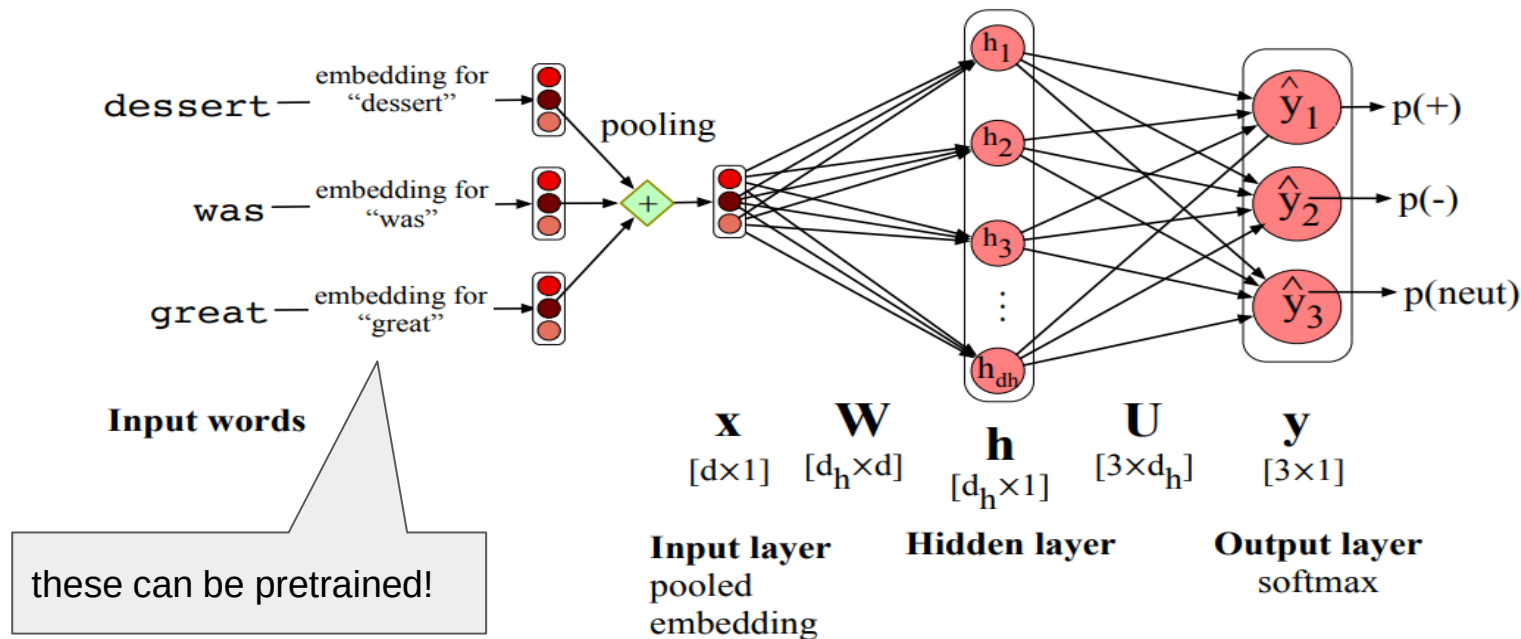
Feed-Forward Neural Networks

- Such neural networks can be readily applied to a range of NLP tasks



Deep Learning

- **Deep Learning** idea - instead of crafting features, we learn from **raw** text - aka **representation learning** - with our word **embeddings**!



One-hot vectors & embeddings

- A vector of length $|V|$
- 1 for the target word and 0 for other words
 - So if “apple” is vocabulary word **5** the **one-hot vector** is
 - $[0\ 0\ 0\ 0\ 1\ 0\ 0\ \dots\ 0\ 0\ 0\ 0]$
 $\quad\quad\quad 1\ 2\ 3\ 4\ 5\ 6\ 7\ \dots\ \dots\ \dots\ |V|$

- Embedding matrix **E**
 - stores embeddings of all words in vocab

The diagram illustrates the process of generating an embedding vector from an embedding matrix and a one-hot vector. On the left, a light blue rectangular matrix labeled **E** represents the embedding matrix. Its height is labeled d and its width is labeled $|V|$. A vertical green strip within the matrix is labeled **5** at the bottom, indicating the column corresponding to the word "apple". This matrix is multiplied (indicated by a large $\times$ symbol) by a one-hot vector. The one-hot vector is a vertical black bar of height $|V|$ with a single white square at the 5th position, labeled **5** to its right. The result of the multiplication is an embedding vector, shown as a vertical green bar of height d labeled $\mathbf{e}_5$ at the bottom. The entire equation is shown as $\text{Matrix } E \times \text{One-hot vector} = \text{Embedding vector } \mathbf{e}_5$.

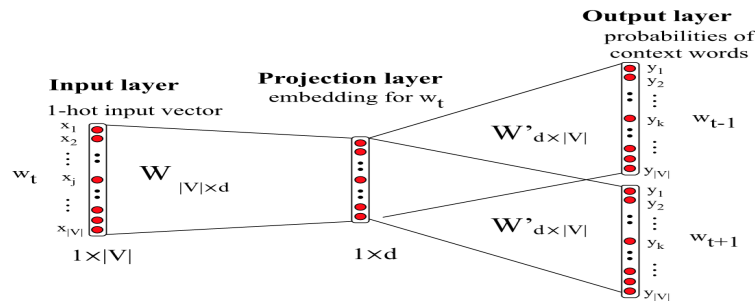
Revisiting word2vec

Word2vec = possibly the simplest “neural” LM

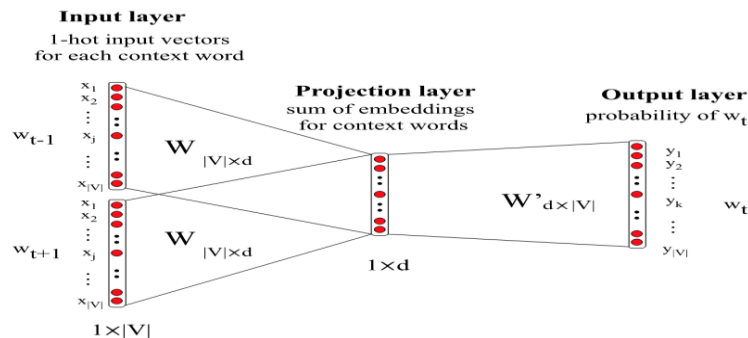
- 2 variants of the model training:

- Skip-gram

- Predict each neighboring word from a given “middle” word



- CBOW (Continuous Bag Of Words)
 - Predict the “missing” middle word from its neighbors



Word2vec skip-gram: visual

